# Anomaly Detection for In-Vehicle Network Using Self-Supervised Learning With Vehicle-Cloud Collaboration Update

Jinhui Cao, *Student Member, IEEE*, Xiaoqiang Di, *Member, IEEE*, Xu Liu, Jinqing Li, Zhi Li, Liang Zhao, *Member, IEEE*, Ammar Hawbani, and Mohsen Guizani, *Fellow, IEEE*

***Abstract*— With the increasing communications between the In-Vehicle Networks (IVNs) and external networks, security has become a stringent problem. In addition, the controller area network bus in IVN lacks security mechanisms by design, which is vulnerable to various attacks. Thus, it is important to detect IVN anomalies for complete vehicular security. However, current studies are constrained by either requiring labeled data or failing to accurately detect message-level anomalies without labeled data. In addition, the concept drift of existing methods has become a challenge over time. To address these problems, this paper proposes an IVN anomaly detection method based on Self-supervised Learning (IVNSL), which is capable of detecting message-level anomalies without labels. The essential idea of IVNSL is to make the message prediction model learn the distribution of normal messages in sequences using message sequences with noise. Furthermore, to accurately detect anomalies, a Message Prediction Model based on Hierarchical transformers (MPMHit) is proposed, which captures the spatial features of the message and the dependencies between messages. Meanwhile, to solve the concept drift over time, this paper proposes an online update mechanism for MPMHit based on vehicle-cloud collaboration. We conduct an extensive experimental evaluation on the car hacking dataset, resulting to an F1-score average and average false positive rates of IVNSL being 2.282% higher and 1.595% lower than the best baseline method. The average detection speed of each message is as fast as 0.1075 ms.**

***Index Terms*— In-vehicle network, controller area network, anomaly detection, self-supervised learning, transformers, vehicle-cloud collaboration.**

## I. Introduction

IN RECENT years, there have been tremendous advances in automotive technology [1]. Intelligent Connected Vehicles (ICVs) can not only connect to infrastructures such as cloud platforms and Road Side Units (RSUs) through Vehicle-to-Infrastructure (V2I) technologies but also share information with passing vehicles through Vehicle-to-Vehicle (V2V) technologies [2]. Despite improving the driving experience with Vehicle-to-Everything (V2X) technologies, the attacked surface of the In-Vehicle Network (IVN) in ICVs is increasing, resulting in IVN being more prone to anomalies and even imperiling the life safety of drivers [3], [4], [5], [6]. In addition, Electronic Control Units (ECUs) in an IVN perform different specific functions in the control system, such as controlling speed and steering, which communicate through the Controller Area Network (CAN) bus protocol. However, the security of the CAN bus was not considered in the design [7], [8], [9]. Hence, it is vulnerable for the CAN in IVNs to be attacked [10]. For example, in 2015, Charlie Miller and Chris Valasek exploited a vulnerability of the Uconnect system in the Chrysler automobile [11]. They remotely sent messages to the CAN bus and started a dangerous event of various functions on the vehicle, leading Chrysler to recall more than 1.4 million vehicles for an upgrade. Anomaly detection can proactively help detect and predict anomalies in advance, thus it has become one of the essential tools for securing IVNs [12].

There are a number of available methods that have the ability to detect anomalies for IVNs, including traditional and deep learning-based methods [13]. Traditional methods usually use IVN characteristics for anomaly detection, such as system specification of the IVN [14], [15], physical attributes of the ECU [16], [17], parameters of the IVN [18], [19], and the information entropy of the network traffic [20], [21]. Nevertheless, these traditional methods need manually to extract

Manuscript received 24 March 2023; revised 8 October 2023; accepted 2 January 2024. Date of publication 27 March 2024; date of current version 2 July 2024. This work was supported in part by the Jilin Science and Technology Development Plan Project of China under Grant 20230508096RC, in part by the Jilin Education Department Project of China under Grant JJKH20220773KJ, in part by the Chongqing Municipal Bureau of Science and Technology Project of China under Grant CSTB2022NSCQ-MSX1434, in part by the National Natural Science Foundation of China under Grant 62372310, in part by the Liaoning Province Applied Basic Research Program under Grant 2023JH2/101300194, and in part by the Liaoning Revitalization Talents Program under Grant XLYC2203151. The Associate Editor for this article was S. Garg. *(Corresponding author: Xiaoqiang Di.)*

Jinhui Cao, Xu Liu, Jinqing Li, and Zhi Li are with the School of Computer Science and Technology, Changchun University of Science and Technology, Changchun 130022, China, and also with the Jilin Province Key Laboratory of Network and Information Security, Changchun University of Science and Technology, Changchun 130022, China (e-mail: cjh@mails.cust.edu.cn; liuxu@cust.edu.cn; lijinqing@cust.edu.cn; 2021101099@mails.cust.edu.cn).

Xiaoqiang Di is with the School of Computer Science and Technology, Changchun University of Science and Technology, Changchun 130022, China, also with the Jilin Province Key Laboratory of Network and Information Security, Changchun University of Science and Technology, Changchun 130022, China, and also with the Information Center, Changchun University of Science and Technology, Changchun 130022, China (e-mail: dixiaoqiang@cust.edu.cn).

Liang Zhao and Ammar Hawbani are with the School of Computer Science, Shenyang Aerospace University, Shenyang 110136, China (e-mail: lzhao@sau.edu.cn; anmande@mail.ustc.edu.cn).

Mohsen Guizani is with the Department of Machine Learning, Mohamed bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates (e-mail: mguizani@ieee.org).

Digital Object Identifier 10.1109/TITS.2024.3351438

the IVN features and ignore the internal information of the CAN messages, resulting in the inability to detect message tampering attacks and the low accuracy of detection [12]. To address such limitations, deep learning techniques have been introduced into IVN anomaly detection in recent years. These methods focus on training deep detection models, including supervised and unsupervised. The IVN anomaly detection methods based on supervised deep learning demand the labeling of a large number of CAN data [22], [23], [24]. However, it is difficult to attain sufficient labeled data because of the private security and support from experts. Moreover, these methods need to update themselves to cope with new anomalies by collecting new labeled data. Despite avoiding labeling massive amounts of CAN data, most of the available methods based on unsupervised deep learning can only detect whether a time window is anomalous or only consider the partial features, resulting in low accuracy in detecting message-level anomalies [25], [26], [27], [28]. Additionally, the phenomenon of concept drift, defined as the unpredictable changes over time in the distribution of data learning by the model or features of the target predicted by the model, precipitates a decline in detection performance in the future. Therefore, this research aims to develop a scheme capable of accurately detecting IVN message-level anomalies without labels and an online updating mechanism that can address concept drift without affecting the running of the detection method.

To overcome the above problems, we propose an IVN anomaly detection method based on Self-supervised Learning (IVNSL) so that the method can detect message-level anomalies without labeling data. IVNSL models anomaly detection as a message prediction task, that is, a message that is correctly predicted according to its context is normal, otherwise it is abnormal. Furthermore, to accurately detect abnormalities, we propose a Message Prediction Model based on Hierarchical transformers (MPMHit), which captures the spatial features of each message and the dependencies between messages by using two layers of transformer block encoders to encode the physical signal sequence of messages and the message sequence, respectively. Meanwhile, to avoid MPMHit concept drift degrading the performance of IVNSL, we design an online update mechanism for MPMHit, which uses new data to retrain the old model on the cloud and replace the old model with the new one on the vehicle. The vehicle also judges whether the MPMHit occurs concept drift based on information entropy of the probability distribution of the MPMHit output. If the information entropy exceeds the given threshold, the vehicle will request the cloud to update the model, which makes the model always match the distribution of data. We conduct an extensive evaluation of IVNSL on the car hacking dataset, which confirms the effectiveness of our anomaly detection method.

In summary, the main contributions of this paper can be listed as follows:

- Since the appearance of the CAN message depends on its context, an IVN anomaly detection method is proposed, which models anomaly detection as a message prediction task and trains the prediction message model in a self-supervised mode. The proposed method does not require labeling massive CAN data and can detect message-level anomalies.
- Further, we propose a message prediction model based on hierarchical transformers, which is used to learn the distribution of normal messages in sequences. Since the proposed model fuses the spatial features of the ID and payload of the message and captures the dependencies between messages, it can accurately predict normal messages and improves the accuracy of detecting anomalies.
- To adapt to the distribution of normal messages in the sequence over time, an online update mechanism of the message prediction model based on vehicle-cloud collaboration is proposed. We conduct an extensive experimental evaluation on the car hacking dataset. Experiments show that the average F1-score and average false positive rate of our anomaly detection method are 2.282% higher and 1.595% lower than those of the best baseline method, respectively. The detection of every message can be accomplished in 0.1075 milliseconds.

The rest of this paper is organized as follows. Section II reviews the work related to IVN anomaly detection. Section III presents the knowledge about the CAN bus. Section IV gives the specific design details of our detection approach, and Section V introduces the online update mechanism of the model. Section VI gives the performance evaluation. Finally, Section VII draws our conclusions.

## II. Related Work

Currently, IVN anomaly detection methods mainly contain traditional and deep learning-based methods [13].

Traditional methods usually use characteristics of IVN for anomaly detection. According to the characteristics used, these methods can be broadly divided into specification-based, fingerprint-based, parameter monitoring-based, and information theory-based [12]. The main purpose of methods based on specification is to detect abnormal behaviors that do not match system specifications such as protocol and frame format [15]. Muter et al. [14] detect attacks by checking the behavior specifications of sensors. Olufowobi et al. [15] proposed SAIDuCANT, which is a real-time specification-based intrusion detection system. However, since these methods depend on the specification, they are not able to identify the attack that an attacker sends the well-modified CAN message following the system specification. Because different ECUs in IVN usually have unique hardware fingerprint information, methods based on fingerprint use the fingerprint information of ECUs to discover anomalies. Cho and Shin [16] proposed a clock-based intrusion detection system, which analyzed ECUs using the deviation between real and reference clock frequencies. Choi et al. [17] used the electrical signal characteristics of each ECU as the fingerprint to perform anomaly detection. However, these methods require that all ECUs of the vehicle must be profiled beforehand, and the ECU and CAN ID affiliation information is not publicly available, resulting in poor generalizability of the detection. Methods based on parameter monitoring can identify anomalies by comparing parameters

such as data transmission frequency, message transmission period, and time interval of IVN. Young et al. [18] proposed a frequency-based intrusion detection system using the frequency characteristics of CAN messages. Lee et al. [19] proposed an intrusion detection method based on the offset rate between request and response messages in the CAN and time intervals. Intrusion detection systems based on information theory can determine anomalous according to network fluctuation. Müter and Asaj [20] identified attacks through the entropy of the collected network traffic compared to the entropy of historical traffic. Marchetti et al. [21] also proposed an entropy-based anomaly detection scheme. Nevertheless, these traditional methods need to manually extract IVN features and ignore the internal information of the message, leading to the failure to detect message tampering attacks and poor accuracy of detection. Therefore, in this paper, to improve the detection accuracy, the spatial features of the message and the interdependencies of messages in the sequence are automatically extracted using hierarchical transformers.

Given that deep neural networks have the capacity to automatically extract features, deep learning has been introduced into IVN anomaly detection in recent years. These methods based on deep learning can roughly consist of supervised and unsupervised according to the learning paradigm. Supervised methods usually use labeled CAN message data to train the deep detection model, and use the trained model for online anomaly detection. Javed et al. [22] proposed a CAN bus intrusion detection approach using the combination of the convolutional neural network and attention-base gate recurrent unit. Flora et al. [23] used data feature vectors and labels of CAN messages to train neural networks and the multi-layer perceptron of the CAN bus attack detection in a supervised manner. Hoang and Kim [24] combined convolutional autoencoder and generative adversarial networks to detect IVN intrusion. However, these supervised learning methods require a lot of expert time and effort to label the data. To avoid massive labeling data, Longari et al. [25] detected anomalies by measuring the difference between the reconstructed sequences and the corresponding original sequences using the Long Short-Term Memory (LSTM) autoencoder. Nam et al. [26] proposed a model containing two bi-directionally connected GPT networks for IVN anomaly detection according to the negative log-likelihood value of the CAN ID sequence. Zhu et al. [27] compared the information of the next CAN message predicted by LSTM with the information of the actual next CAN message to determine normal or abnormal, but the method ignores the possible data correlation between CAN IDs, which affects the accuracy of detection. Song and Kim [28] proposed a self-supervised anomaly detection method for IVN based on noisy pseudo-normal data, but this method cannot accurately generate pseudo-normal message data, resulting in low detection accuracy. To sum up, these unsupervised IVN anomaly detection methods focus only on the message ID or data domain, and most of them can only detect sequence-level anomalies and can not determine whether each message in a sequence is anomalous. In addition, the detection models of these deep learning methods will occur concept drift over time, which makes the detection accuracy of the model rapidly decline in the future. Thus, to detect message-level anomalies and avoid labeling massive data, this paper proposes an IVN anomaly detection method using self-supervised learning, which makes MPMHit learn the distribution of normal messages in sequences using message sequences with noise. Moreover, to solve concept drift, we propose an online update mechanism of MPMHit based on vehicle-cloud collaboration.

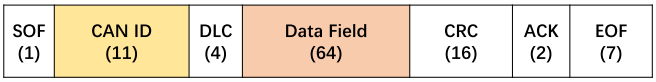


Fig. 1. The structure of the CAN bus data frame.

Compared with existing work about IVN anomaly detection, the performance advantages of our proposed scheme are as follows: 1) It is based on self-supervised learning and does not require labeled data. 2) It can detect message-level anomalies. 3) It takes into account the spatial features of the message and the dependencies between messages and can detect anomalies more accurately. 4) It can be updated online to avoid concept drift.

## III. Preliminaries

The CAN that has high stability and efficiency solves the problems of increased wiring costs, complexity, and weight caused by the increasing number of electronic devices in a vehicle [29]. It has become the factual standard for communication systems in modern vehicles [30]. The CAN controls the transmission of various information between ECU nodes. Each node sends messages to the bus in the predefined frame format using the priority-based arbitration mechanism [31]. Each node can read any data packet from the bus.

The CAN bus standard is divided into two versions according to the number of bits occupied by the arbitration field [32], [33], [34]. The CAN bus 2.0A defines the standard frame format with an arbitration field of 11 bits, while the CAN bus 2.0B not only defines an extended frame format with an arbitration field of 29 bits arbitration field but also is compatible with standard frames. The structure of the CAN data frame is shown in Fig. 1. The start of frame (SOF) field marks the start of the CAN message, with a length of 1 bit. The ID field is the unique identifier of the message, with a length of 11 bits (29 bits for expansion mode). In addition, the ID field also represents the frame priority of arbitration, and the lowest value represents the highest priority. The data length code (DLC) indicates the length of the actual data. The length of this field is 4 bits, ranging from 0 to 8, and is in bytes. The data field carries the communication payload between ECUs, which can be 0 to 64 bits. In this field, the actual semantics of the payload depends on the vehicle manufacturer, which is different from vehicle to vehicle and is not disclosed. The Cyclic Redundancy Code (CRC) field with 16 bits contains a checksum, which is used to detect errors in CAN messages. The acknowledgment (ACK) field with 2 bits is used to confirm whether the message is received correctly. The end of frame (EOF) marks the end of the frame, with a length of 7 bits.

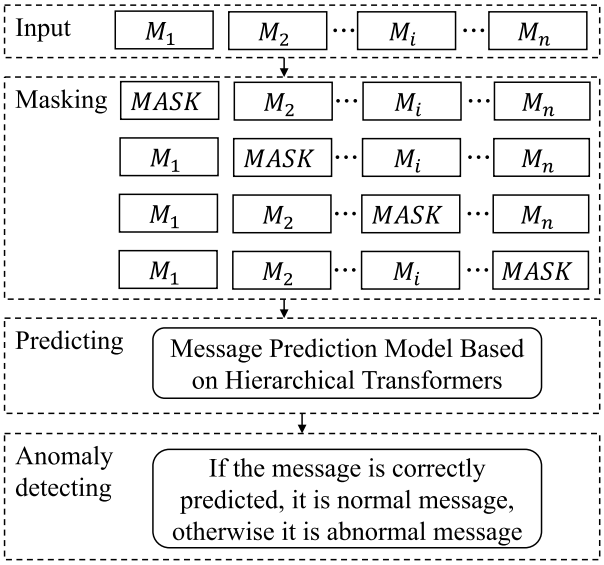


Fig. 2. The overview of the proposed IVN anomaly detection method.

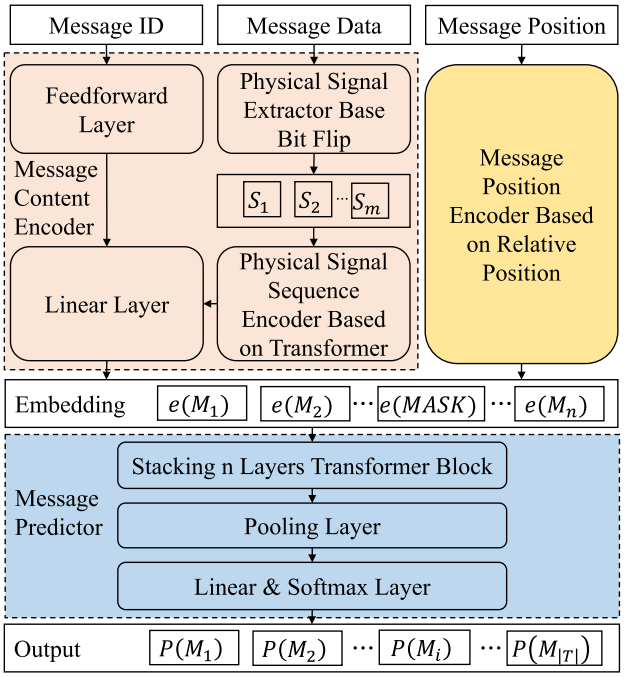


Fig. 3. The architecture of the message prediction model based on hierarchical transformers.

## IV. Anomaly Detection Method

To accurately detect message-level anomalies without using labels, this paper proposes an IVN anomaly detection method named IVNSL. The overview of IVNSL is shown in Fig. 2, with three main modules:

(1) *Masking*: The original message sequence is constructed as the message sequence with noise by masking the message in the sequence.

(2) *Predicting*: The masked messages are predicted using the message prediction model based on hierarchical transformers.

(3) *Anomaly detecting*: If the masked message is correctly predicted, it is a normal message; otherwise, it is an abnormal message.

The intuition behind IVNSL is message appears dependent on its context. Specifically, first, we benefit from the masking language model and generate message sequences with noise by masking the message of a normal sequence. Afterward, in the training phase, message sequences with noise are used as the input of the message prediction model, and original normal message sequences are used as the labels for the self-supervised training of the message prediction model. In the online detection phase, IVNSL uses the trained message prediction model to predict the masked message and judge whether the message is abnormal according to the prediction confidence, that is, if the prediction probability exceeds the threshold, the message is normal, otherwise abnormal.

### A. Masking

The intuition behind the detection method proposed in this paper is that anomalous messages affect the digital vector representation of the entire message sequence. Therefore, we use the sliding window method to construct a fixed length message sequence $M = \{M_1, M_2, \ldots, M_n\}$, where $M_i$ denotes the $i$-th message in the sequence, and n represents the length of the message sequence, i.e. the size of the window. Then, the masking language model [35] is used to mask the message sequence to obtain a noisy message sequence, that is, select a message from the sequence and replace it with a special message $MASK$. For example, the original message sequence is $M = \{M_1, M_2, M_3\}$, and the masked message sequences are $\{MASK, M_2, M_3\}$, $\{M_1, MASK, M_3\}$, $\{M_1, M_2, MASK\}$. This paper sets the ID of the $MASK$ message to 0 and the data field to 0. The masked message sequence is used as the input of the Predicting module, and the masked message is used as the prediction target.

### B. Predicting

In this paper, we propose a message prediction model based on hierarchical transformers (MPMHit), which is a self-supervised model, including the training phase and the prediction phase. In the training phase, masked message sequences are used as the input and original message sequences are used as labels. The model parameters are adjusted by backpropagation to reduce the loss value, while the optimal hyper-parameters are selected. In the prediction phase, the message sequences are predicted by model forward propagation to predict the masked messages. Fig. 3 depicts the complete architecture of MPMHit, where $|T|$ represents the number of types of all messages. First, MPMHit uses a message position encoder based on the relative position and a message content encoder based on the transformer to extract the spatial features of the position and content of the message, respectively and fuses them to obtain the embedding of each message. Then, MPMHit uses a message predictor based on the transformer to capture the dependencies between messages in the sequence and predict the masked message.

*1) Position Encoder:* Considering the relative positions of messages in the sequence, this paper designs a message position encoder based on sine and cosine functions to calculate

the vector $MP_i \in R^d$ for the relative position of each message in the sequence,

$$MP_i = \begin{cases} \sin(\frac{i}{10000^{\frac{j}{d}}}), & if \ \ j\%2 = 0 \\ \cos(\frac{i}{10000^{\frac{j}{d}}}), & if \ \ j\%2 = 1 \end{cases}. \quad (1)$$

Here, $i = 1, 2, \ldots, n$ is the position index of each message in the sequence, $j = 0, 1, \ldots, d-1$ is the index of the elements in the vector $MP_i$. In addition, the sine and cosine functions can be used interchangeably, and these two functions have an approximately linear dependence on the position parameter $i$.

*2) Content Encoder:* Since the CAN ID is the unique identifier of the message and indicates the frame priority of arbitration, and the CAN data field represents the actual payload data carrying the physical signals of the vehicle, this paper proposes a CAN message content encoder based on the transformer, which fuses the spatial features of the ID and data fields, as shown in Fig. 3. The module consists of three components: a physical signal extractor based on bit-flip, a physical signal sequence encoder based on the transformer, and a fusion of the spatial features of the ID and payload.

*a) Physical signal extractor based on bit-flip:* Considering that the signals of physical functions of actual vehicles are encapsulated in the payload data and that encoding the payload data directly causes a large amount of computational consumption, the bit flip rate method [36] is used to extract the boundaries of the physical signals in messages. This method has the ability to fragment payload data into several physical signals, the quantity of which is significantly less than the payload's length. By using these physical signals as inputs for the deep anomaly detection method, computational consumption can be substantially reduced. Specifically, in a sequence of consecutive messages with the same CAN ID, the bit-flip rate array $B \in R^l$ is calculated by Eq.(2), and $l$ is determined by the payload length $DLC$ of a given message.

$$B_i = \frac{bitFlip\_num_i}{\#all\_mun}. \quad (2)$$

Here, $i = 1, 2, \ldots, 8 \times DLC$ is the position index of the element in the array $B$, $bitFlip\_num_i$ indicates the number of bit flips occurred at the $i$-th position of the data field, $\#all\_mun$ represents the total number of messages in the sequence of consecutive messages with the same CAN ID.

We use Eq.(3) to calculate the magnitude array $MA \in R^l$, whose each element represents the magnitude of the bit-flip rate of the corresponding bit in the data field.

$$MA_i = \lceil \log_{10}(B_i) \rceil. \quad (3)$$

Here, $M_i$ and $B_i$ are the $i$-th element of the magnitude array and the bit-flip array, respectively.

Then, we scan the magnitude array for the successive bit couples where the bit-flip magnitude of the first bit is higher than the bit-flip magnitude of the second bit. Whenever a similar pair occurs, a boundary is set between the two bits. Finally, the physical signals sequence $S = \{s_1, s_2, \ldots, s_m\}$ for each message is generated, where $s_i$ is the physical signal intercepted according to the extracted boundary, and $m$ represents the number of physical signals in each message.

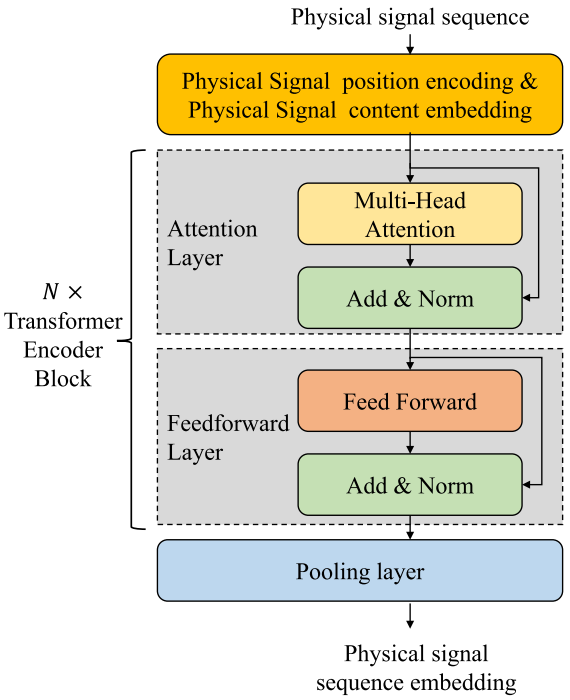


Fig. 4. Physical signals sequence encoder based on the transformer.

*b) Physical signals sequence encoder based on transformer:* In this paper, we use the transformer [37] to capture the dependencies between physical signals in a sequence $S$ and convert the sequence $S$ into a fixed dimensional vector $SV \in R^d$, as shown in Fig. 4.

First, the relative position vector $SP \in R^d$ of each physical signal in the sequence $S$ is computed using the position encoder in Section IV-B.1, and we use one-hot [38] to encode the content of each physical signal as vectors $SC_{one} \in R^{|K|}$, where $|K|$ is the total number of categories of physical signals, and use $SC_{one}$ to find the representation vector $SC$ in the learnable weight matrix $W_{SC} \in R^{|K| \times d}$, i.e.

$$SC = SC_{one} \times W_{SC}. \quad (4)$$

Here, $d$ is the dimension of the physical signal content representation vector SC, $|K| \gg d$.

The two vectors can be added as $SX = SP + SC$ to represent the embedding of each physical signal. Thus, the physical signal sequence $S = \{s_1, s_2, \ldots, s_m\}$ is converted into a matrix $SX'$ of dimension $m \times d$,

$$SX' = \{SX_1, SX_2, \ldots, SX_m\}. \quad (5)$$

Then, we input $SX'$ into $N$ stacked transformer encoder blocks, where each transformer encoder block includes two sub-layers: the attention layer and the feedforward layer.

In the attention layer, we use a multi-headed self-attention mechanism, and the self-attention of each head is calculated as follows.

$$Z_l = Attention(Q_l, K_l, V_l) = softmax(\frac{Q_l \times K_l^T}{\sqrt{\omega}})V_l. \quad (6)$$

Here, $l \in 1, 2, \ldots, L$ denotes the number of attention heads, $\omega$ is an integer satisfying $\omega \times L = d$, and the parameters $Q_l$, $K_l$, and $V_l$ are the query matrix, key matrix, and value matrix, respectively, which are obtained by multiplying $SX'$ by three learnable weights $W_Q, W_K, W_V \in R^{d\times\omega}$, respectively, as follows:

$$Q_l = SX' \times W_Q, K_l = SX' \times W_K, V_l = SX' \times W_V. \quad (7)$$

Dividing by $\sqrt{\omega}$ stabilizes the gradient during the training process.

These attention heads are concatenated and then multiplied by an additional weight matrix $W_O$ to obtain a multi-headed self-attention matrix $Z \in R^{m\times d}$,

$$Z = MultiAttention(SX') = concat(Z_1, \ldots, Z_L) \times W_O. \quad (8)$$

Here, $W_O$ is a matrix of dimension $d \times d$.

The residual connection is added to the layer and layer normalization is performed as follows:

$$Z' = layernorm(Z + SX'). \quad (9)$$

After that, the output of the self-attention layer is fed to the feedforward layer. The feedforward layer consists of a linear layer with a non-linear activation function and a normal linear layer to generate a matrix of size $m \times d$ as follows:

$$Z'' = GELU(Z' \times W_1 + b_1) \times W_2 + b_2. \quad (10)$$

Here, $W_1$ and $W_2$ are matrices of dimensions $d \times 4d$ and $4d \times d$, respectively, $b_1$ and $b_2$ are matrices of dimensions $m \times 4d$ and $m \times d$, respectively, and $GELU(\cdot)$ is the Gaussian error linear unit activation function [39].

Then the matrix $Z''$ is added to the residual connection and layer normalized to generate a matrix of size $m \times d$, denoted as,

$$Z''' = layernorm(Z'' + Z'). \quad (11)$$

Finally, the output of the $N$-th transformer encoder block is fed to the average pooling layer, which converts the physical signal sequence matrix into a fixed-dimensional vector, each element of which is calculated as follows:

$$SV_i = \frac{1}{m}\sum_{j=0}^{m} Z'''_{j,i}. \quad (12)$$

Here, $i$ denotes the position of the element of vector $SV$ and $Z'''_{j,i}$ denotes the value of the $j$-th row, and the $i$-th column of matrix $Z'''$.

*c) Fusion of the spatial features of the ID and payload:* In this paper, we input the message ID $X'_I \in \{0, 1\}^r$ into the feedforward layer to extract the spatial features of ID and generate the ID embedding vector $IV$ with size $d$, as shown below:

$$IV = GELU(X'_I \times W_1^I + b_1^I) \times W_2^I + b_2^I. \quad (13)$$

Here, $r$ is the bit length of the CAN ID, $W_1^I$ and $W_2^I$ are matrices of dimensions $r \times 4d$ and $4d \times d$, respectively, and $b_1^I$ and $b_2^I$ are vectors of dimensions $4d$ and $d$, respectively.

Then the spatial features of the ID and payload are fused using the linear layer to obtain the vector embedding $MC \in R^d$ of the message content as follows:

$$MC = (SV + IV) \times W_{MC} + b_{MC}. \quad (14)$$

Here, $W_{MC}$ is a matrix of dimension $d \times d$, and $b_{MC}$ is a vector of dimension $d$.

*3) Message Predictor:* In this paper, the position vector and content vector of each message in the message sequence $M$ are summed to obtain the message vector embedding $MX \in R^d$, as follows,

$$MX = MP_i + MC. \quad (15)$$

Thus, the message sequence $M$ is transformed into a matrix with dimensions $n \times d$:

$$MX' = \{MX_1, MX_2, \ldots, MX_n\}. \quad (16)$$

The dependencies between messages in the sequence $M$ are captured using the stacked $N$ layer transformer to obtain the multi-headed self-attention matrix $MX' \in R^{m\times d}$:

$$MSM = MultiAttention(MX'). \quad (17)$$

Then, the vector embedding $MX'' \in R^d$ of the message sequence $M$ is obtained using average pooling, as follows,

$$MX''_i = \frac{1}{m}\sum_{j=0}^{m} MSM_{j,i}. \quad (18)$$

Here, $i$ denotes the position of the element of vector $MX''_i$ and $MSM_{j,i}$ denotes the value of the $j$-th row and the $i$-th column of the matrix $MSM$.

Finally, the probability distribution vector $P \in R^{|T|}$ of the messages is obtained using the linear and softmax layer:

$$P = softmax(MX'' \times W_{MS}) + b_{MS}. \quad (19)$$

Here, $W_{MS}$ is a matrix of size $d \times |T|$, $b_{MS}$ is a vector of size $|T|$.

During training, the predicted messages are used as labels for self-supervised learning.

### C. Anomaly Detecting

After MPMHit is trained, anomaly detection is performed online. Therefore, we feed a sequence of CAN messages into MPMHit and configure the masking module in such a way that each message is masked successively, one at a time. We measure the ability of MPMHit for predicting each message to determine whether the message is abnormal. A high confidence in the prediction of a message indicates normal, while a low confidence is interpreted as an anomaly. More specifically, we use the following procedure. If the prediction of a message lies in the first $\varepsilon$ predictions, we consider it a normal message, otherwise, it is considered an abnormal message.

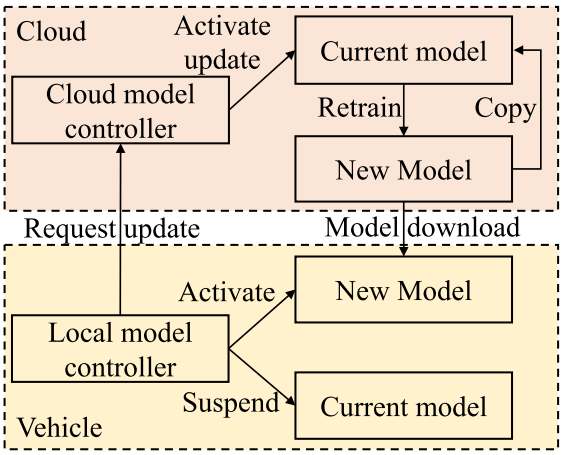


Fig. 5. Online update mechanism.

## V. Online Update Mechanism

The distribution of normal messages in the current and historical sequences deviates over time, which leads to the concept drift of MPMHit. The concept drift may cause the most normal messages to be falsely detected. Thus, it is essential to update the proposed message prediction model so that it always conforms to the distribution of time-varying data. However, in the existing IVN anomaly detection methods based on deep learning, once the trained deep model has been installed on the vehicle, it cannot be adaptively retrained due to the limitations of the computing power of the vehicle. The vehicle-cloud collaborative mechanism addresses this issue by retraining the deep model on the cloud with powerful computing power and installing the retrained model on the vehicle, which greatly liberates the computing resources of the vehicle and makes the model updating workable. Therefore, this paper proposes an online update mechanism for MPMHit based on vehicle-cloud collaboration, as shown in Fig. 5, which consists of two parts: the cloud side and the vehicle side. The cloud side retrains the model and the vehicle side deploys the new model. Specifically, when the model occurs concept drift, the local model controller requests the cloud model controller to update the model. Then, the cloud model controller activates the model update, which retrains the current model using the new data to obtain the new model and sets the copied new model as the current model on the cloud. The vehicle side downloads the new model locally. Finally, the local model controller activates the new model and hangs the current model simultaneously. The new model is set as the current model and the old model is deleted.

The local controller on the vehicle side should provide a triggering strategy for MPMHit updates. The commonly used triggering update strategies are time-triggering and event-triggering. Timed strategies usually update the model at a given fixed time interval. In practice, the moment of model concept drift is often uncertain. Therefore, it is arduous for time-triggered strategies to determine the exact moment of occurrence of the model concept drift. If the time interval is set too small, the model is frequently updated leading to a waste of resources, and conversely, the detection model on the vehicle is frequently false negative. The event-triggered strategy triggers model updates based on whether a model concept drift event has occurred. Since this approach can detect the exact moment of occurrence of the model concept drift and trigger the model update timely, it is better in practice. In addition, given a sequence of masked messages, the output of MPMHit is a probability distribution of all kinds of messages. The information entropy of the probability distribution of the MPMHit output also increases significantly when the model concept drift causes the anomaly detection performance to deteriorate. Therefore, an event-triggering strategy based on information entropy is proposed in this paper.

From Section IV-A, the sampled message sequence $M = \{M_1, M_2, \ldots, M_n\}$ can generate $n$ masked message sequences. Therefore, the average information entropy $\bar{q}$ can be calculated by the following equation:

$$\bar{q} = \frac{1}{n} \sum_{i=0}^{n} q_i. \tag{20}$$

Here, $q_i$ denotes the information entropy of the probability distribution of the output of MPMHit whose input is the $i$-th masked message sequence generated by $M$. $q_i$ is calculated as follows:

$$q_i = -\sum_{j=0}^{T} P_j \times \ln P_j. \tag{21}$$

Here, $P_j$ is the $j$-th element of the probability distribution vector $P \in R^T$ and $0 < q_i < \ln T$.

We first calculate the average information entropy $\bar{q}$ using Eq.(20) and Eq.(21). A threshold $\delta$ is set for $\bar{q}$ to measure whether the model undergoes concept drift. If $\bar{q}$ is greater than the threshold $\delta$, the local model controller triggers a model update request.

## VI. Performance Evaluation

### A. Experimental Datasets

We use the car hacking dataset [40]. This dataset consists of a normal CAN traffic file and four attacked CAN traffic files. The four attacked traffic files correspond to four types of attacks, namely Denial-of-service (DoS) attack, fuzzy attack, spoofing the drive gear attacks, and spoofing the engine RPM. DoS attack is executed by inserting a message with the CAN ID '$0 \times 00$' into the CAN bus. Fuzzy attacks are executed by inserting messages with random CAN IDs. Spoofing the drive gear attack and spoofing the engine RPM are executed by injecting messages with CAN IDs related to engine speed and transmission, respectively. Thus, each attacked dataset contains normal messages and attacked messages injected by the attacked node.

In this paper, the messages in the normal CAN traffic file and the normal messages in the 4 attacked CAN traffic files are used as the training set to train the model. we extract 10000 consecutive CAN messages from each of the four attacked CAN communication files as a test set to evaluate the effectiveness of IVNSL.

### B. Evaluation Metrics and Experimental Environment

Since anomalies usually occur by chance, resulting in a possible imbalance between normal and anomalous messages, we use Precision, Recall, and F1-score [41] to evaluate the performance of IVNSL. In addition, the False Negative Rate (FNR) is very important in IVN anomaly detection, which can affect the user experience if a high false negative rate is generated [42]. These metrics are calculated as:

$$Precision = \frac{TP}{TP + FP} \tag{22}$$

$$Recall = \frac{TP}{TP + FN} \tag{23}$$

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{24}$$

$$FNR = \frac{FN}{TP + FN}. \tag{25}$$

Here, True positive (TP) indicates the number of actual abnormal messages correctly detected as abnormal messages, False positive (FP) indicates the number of actual normal messages incorrectly detected as abnormal messages, and False negative (FN) indicates the number of actual abnormal messages incorrectly detected as normal messages.

The Area Under the Curve (AUC) value, which can determine the sensitivity of detection methods to the threshold, is calculated by standardizing the area below the Receiver Operating Characteristic (ROC) curve. ROC curve is constructed by plotting the False Positive Rate (FPR) and True Positive Rate (TPR) values corresponding to each threshold on the horizontal and vertical axes of the two-dimensional graph respectively, where $TPR = Recall$ and $FPR$ is calculated as:

$$FPR = \frac{FP}{FP + TN}. \tag{26}$$

In addition, we conducted all experimental evaluations on Ubuntu 18.04.5 LTS, which has Intel (R) Xeon (R) W-2123 3.60GHZ CPU, NVIDIA TITAN Xp GPU, and 64 GB RAM. For developing the method, we used Pytorch for the deep learning model.

### C. Experimental Results

In this subsection, we compare IVNSL with two available traditional methods: OTIDS [19], SAIDuCANT [15], one supervised method Supervision [23], one unsupervised method MT-LSTM [27], and one self-supervised method Supervision [28] using the performance metrics in Section VI-B.

Fig. 6 shows IVNSL compared with other baseline methods in terms of Precision. The detection Precision of our method for DoS attacks is 100%, which is 47%, 8.7%, 2.5%, and 2.29% higher than MT-LSTM, Supervision, Self-Supervision, and SAIDuCANT respectively. Since MT-LSTM and Supervision ignore the temporal features of the messages, while our method focuses on the dependencies of the messages in the sequence and extracts the temporal features, our method is significantly better Precision of our method than MT-LSTM and Supervision. Since SAIDuCANT cannot detect sending a large number of attack messages that conform to the system specification, and Self-Supervision cannot accurately model the distribution of attacked messages when generating pseudo-normal message data, the Precision of our method is slightly better than SAIDuCANT and Self-Supervision, respectively. The detection Precision of our method for fuzzy attacks is 100%, which is 34%, 13.61%, and 5.54% higher than MT-LSTM, SAIDuCANT, and Self-Supervision respectively because our method can capture the spatial features of message ID and payload, while MT-LSTM ignores the spatial features of subsequent messages, SAIDuCANT ignores the content spatial features of messages, and Self-Supervision ignores the payload of messages and can only detect sequence-level anomalies. Although the results of the Precision of our method and Supervision are similar on the fuzzy attack dataset, Supervision needs label data. In addition, if Supervision has concept drift, its detection performance will deteriorate sharply, which needs to train the model with new label data. Labeling massive CAN messages will waste a lot of time and experience of experts. On the contrary, in our scheme, if the vehicle detects a concept drift, the local model controller in the vehicle immediately requests the cloud to update the model, which does not need to label the data. The detection Precision of our method for spoofing gear and RPM attacks is 97.73% and 100%, respectively. The Precision of IVNSL on the two spoofing attack datasets is significantly better than that of SAIDuCANT, MT-LSTM, and Self-Supervision, with an average of 17.55%, 32.8%, and 1.39% higher, respectively. This is because our method can capture both the spatial features of the message and the dependencies of messages in the sequence. Although OTIDS and Supervision are slightly more accurate than IVNSL on the two spoofing attack datasets, our method does not require label data to detect spoofing attacks of the message level.

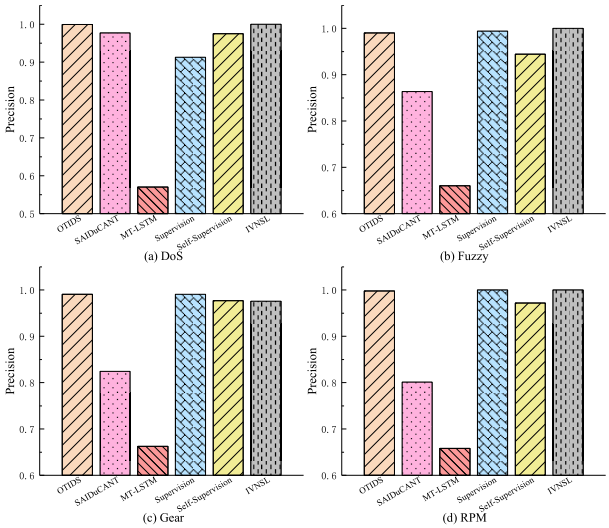


Fig. 6. IVNSL compared with other baseline methods in terms of precision.

Fig. 7 shows IVNSL compared with other baseline methods in terms of Recall. IVN has a 100% Recall for detecting DoS attacks. Our method is slightly better than Self-Supervision, close to SAIDuCANT, and significantly better than OTIDS, MT-LSTM, and Supervision by 26.2%, 41.87%, and 11.4%,

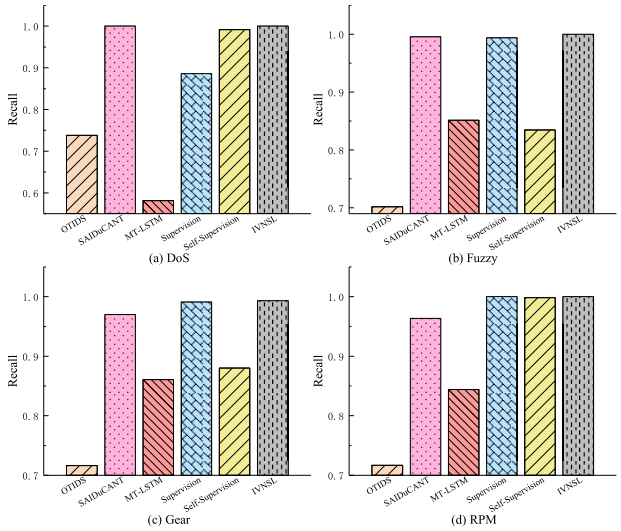


Fig. 7. IVNSL compared with other baseline methods in terms of recall.

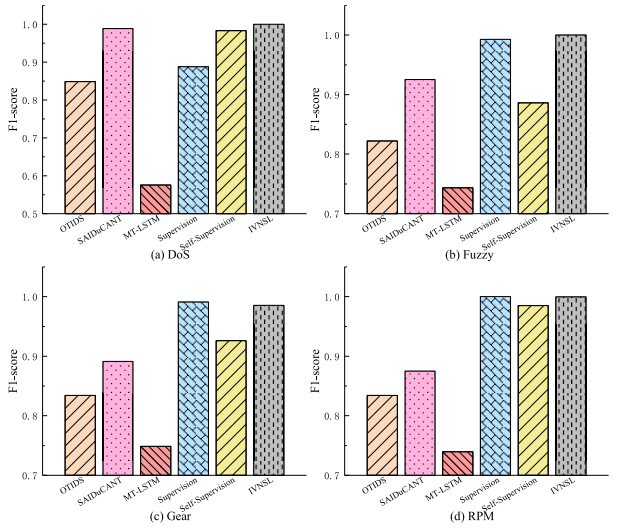


Fig. 8. IVNSL compared with other baseline methods in terms of F1-score.

respectively, because our method can learn the dependencies between messages and extract the temporal features of messages, while the interval-based OTIDS tends to incorrectly detect a large number of attack messages that match the time interval as normal messages, MT-LSTM ignores the time information of following messages, and Supervision does not focus on the temporal features of messages. Our method detects 100% Recall for fuzzy attacks, and IVNSL slightly outperforms SAIDuCANT and Supervision. In addition, IVNSL's Recall is significantly better than OTIDS, MT-LSTM, and Self-Supervision, by 29.79%, 14.87%, and 16.55%, respectively, because our method can capture the spatial features of the messages and the dependencies between messages, while OTIDS and Self-Supervision ignore the internal spatial features of the payload in the CAN message, MT-LSTM ignores the spatial features of subsequent messages. Moreover, Self-Supervision can only detect sequence-level anomalies. Due to the fact that spoofing attacks are executed by injecting messages related to vehicle functionality, the attack is related to the spatiotemporal features of the messages. OTIDS and SAIDeCANT can only capture the temporal features, ignoring the content spatial features of messages. MT-LSTM ignores the spatial features of the subsequent messages, Self-Supervision ignores the spatial features of the payload of messages. Our method can learn the spatiotemporal features of the messages simultaneously. Thus, The detection Recall of our method for spoofing gear and RPM attacks is 99.34% and 100%, respectively, which significantly outperforms OTIDS, SAIDuCANT, MT-LSTM, and Self-Supervision on average with a higher Recall of 28%, 2.98%,14.32%, and 5.74%. Although the IVNSL and Supervision Recall results are similar, our method does not require labeled data.

The anomaly detection F1-score can balance the detected Precision and Recall. Fig. 8 shows IVNSL compared with other baseline methods in terms of F1-score. DoS attacks typically prevent normal communication between ECUs by injecting a large number of CAN messages in a short period. Our method can extract the dependency relationship between packets, i.e. time features, and improves the F1-score for detecting Dos attacks. Thus, IVNSL detects DoS attacks with an F1-score of 100% and our method significantly outperforms OTIDS, MT-LSTM, and Supervision by 15.12%, 42.44%, and 11.2%, respectively, and slightly outperforms SAIDuCANT and Self-Supervision by 1.16% and 1.67%, respectively. Fuzzy attacks are performed by inserting messages with random CAN IDs and payloads. Our method can extract the spatial features of the CAN ID and the payload of the message, thus accurately detecting fuzzy attacks. Therefore, IVNSL detects fuzzy attacks with a detection F1-score of 100% and our method significantly outperforms OTIDS, SAIDuCANT, MT-LSTM, and Self-Supervision by 17.8%, 7.48%, 25.64%, and 11.39%, respectively, and slightly outperforms Supervision by 0.7%. Spoofing the drive gear attack and spoofing the engine RPM are executed by injecting messages with CAN IDs related to engine speed and transmission, respectively. Our method fuses the spatial features of the ID and payload of the message and captures the dependencies between messages, improving the F1-score for detecting spoofing attacks. Thereby, IVNSL significantly outperforms OTIDS, SAIDuCANT, MT-LSTM, and Self-Supervision, on average, by 15.82%, 10.93%, 24.83%, and 3.69%, respectively, for the two spoofing attack datasets. Although the F1-score of IVNSL and Supervision is similar, our method does not require labeled data. In addition, the average F1-score of our method on the four attacked test datasets is 2.82% higher than the best baseline method.

An excellent IVN anomaly detection method should have a low FNR for anomalies at the message level. Fig. 9 shows IVNSL compared with other baseline methods in terms of box-plot of FNR and line chart of average FNR. It can be observed that OTIDS has the highest FNR at the median and mean values, while IVNSL has the lowest FNR at the median and mean values. Although most baseline methods achieve an FNR of below 20% for a given attack dataset, when applied to all given attacks, they differ significantly. IVNSL outperforms all other baseline methods in terms of FNR with a minimum

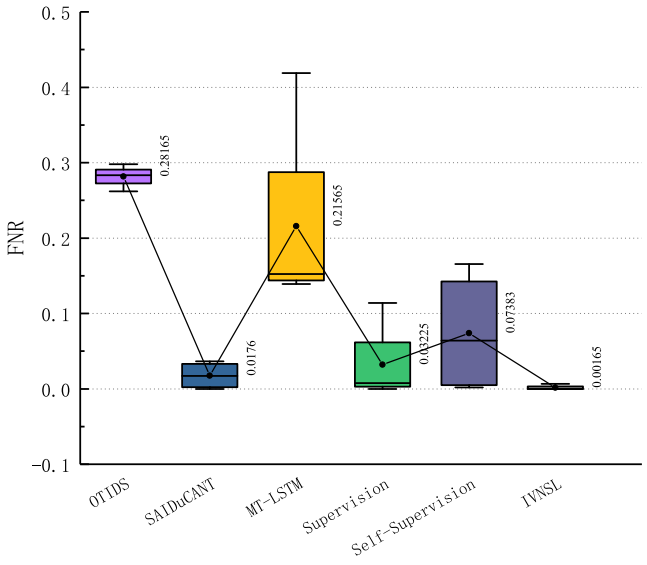


Fig. 9. IVNSL compared with other baseline methods in terms of FNR.

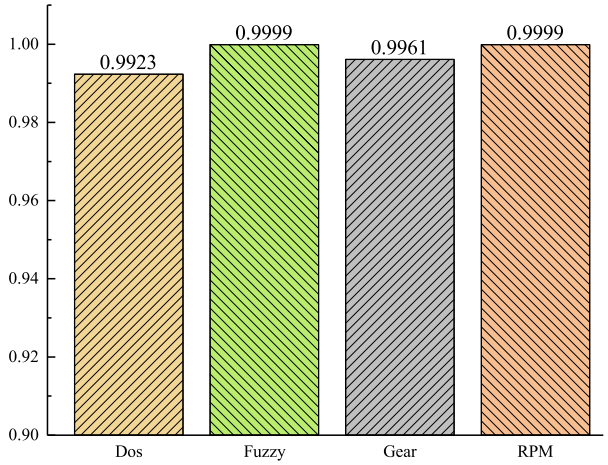


Fig. 10. The AUC of IVNSL.

value of 0%, a median value of 0%, and a mean value of 0.00165%, where the mean value is 1.595% lower than the best method. Our method has the best FNR for detecting anomalies at the message level due to the following reasons. First, our method utilizes a self-supervised approach for message prediction, which can detect message-level anomalies. Second, our message prediction model captures both the spatial features of the message and the dependencies between messages.

Since the closer the AUC value is to 1, the less sensitive the detection method is to the selection of the threshold, AUC is used to visually demonstrate the sensitivity of IVNSL to the threshold $\varepsilon$ for anomaly detection, where the threshold $\varepsilon$ is selected in the interval $[1, |T|]$. As shown in Fig. 10, the AUC values of IVNSL for Dos, fuzzy, gear spoofing, and RPM spoofing attacks are 0.9923, 0.9999, 0.9961, and 0.9999, respectively. This indicates that IVNSL has low sensitivity to the threshold. IVNSL judges anomalies based on the accuracy of the message by predicted, i.e. if the prediction of a message lies in the first $\varepsilon$ predictions, we consider it a normal message, otherwise, it is considered an abnormal message. Since the

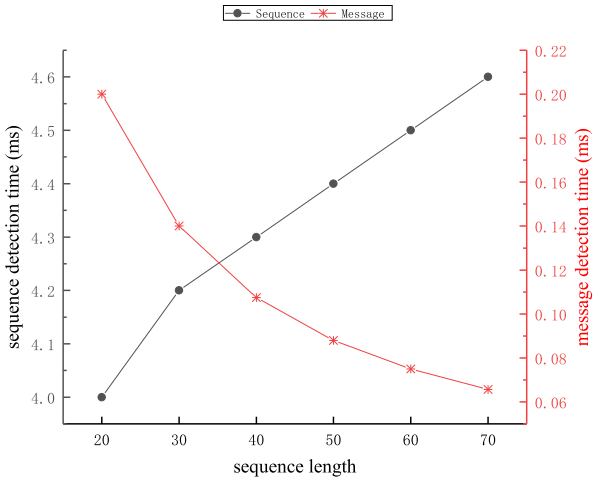


Fig. 11. The testing of average time cost on the sequence and the message.

proposed MPMHit fuses the spatial features of the ID and payload of the message and captures the dependencies between messages, it can accurately predict normal messages. In other words, if the message is normal, the probability of being predicted is high, while if the message is abnormal, the probability of being predicted is low. Therefore, IVNSL is not sensitive to the selection of the threshold $\varepsilon$.

### D. Efficiency Evaluation

Since IVN is a time-critical network, the detection latency is a key indicator for evaluating detection methods. The detection delay is defined as the time from the appearance of an anomaly message to the detection of an anomaly message, as shown in the following equation:

$$t_l = t_d - t_a \quad (t_{cs} \le t_a \le t_{ce}). \tag{27}$$

Here, $t_d$ indicates the moment when the anomaly is detected, and $t_a$ represents the moment when the anomaly occurred. $t_{cs}$ and $t_{ce}$ are the start and end moments of the collected sequence of input messages. In addition, $t_d$ is the sum of $t_{ce}$, $t_{dp}$ and $t_i$, where $t_i$ represents the inference time of the proposed message prediction model, and $t_{dp}$ represents the generation time delay of the masked message sequence. Therefore, the detection latency $t_l$ is also calculated as follows:

$$t_l = t_{ce} + t_{dp} + t_i - ta \quad (t_i + t_{dp} \le t_l \le \triangle t + t_i + t_{dp}). \tag{28}$$

Here, $\triangle t = t_{ce} - t_{cs}$. Obviously, given $t_{cs} = t_a$ or $t_{ce} = t_a$, detection latency gets the maximum or minimum values. For the purpose of real-time detection, all latencies should be minimized. Since $n$ masked message sequences generated by message sequences with length $n$ are input into MPMHit as a batch, data collection time $\triangle t$ and the processing time $t_{dp} + t_i$ depend on the length of the message sequence. Therefore, we give the test time of different message sequence lengths, as shown in Fig. 11.

For detecting the whole message sequence, a significant trend is that the time cost increases with the increase in message length. Undoubtedly, this is due to the comprehensive

factors of masked message sequence construction and message prediction. Since this model predicts that each masked message is equivalent to detecting each message, it also reports the average time cost result of a single message. However, it shows the contrary trend to the time cost of the whole message sequence, which can be attributed to redundant message processing and continual detection inference. Therefore, an eclectic solution should be selected (for example, the message sequence length is 40) to reduce the anomaly detection delay. We can observe that when the length of the message sequence is 40, the time cost of the proposed method to detect the whole message sequence and the average time cost of each message are 4.3 ms and 0.1075 ms, respectively. This means that this method can infer 9302 messages per second. Since the CAN bus of real vehicles transmits about 2000 messages per second, the proposed method has sufficient capacity for real-time detection.

### E. Discussion

The vehicle generates approximately 2000 CAN messages per second, that is, a CAN message is generated every 0.5 ms in IVN [40]. In addition, since the upload speed of 5G is 100 Mbps [43], and the size of each message is 105 bits, the average transmission delay for each message is 0.00105 ms, which is less than 0.5 ms. Therefore, real-time transmission of messages to the cloud is possible, and the online model update mechanism based on vehicle-cloud collaboration is feasible in practice. The delay of retraining models on the cloud is related to the computing power and number of CAN messages. Assuming that Ubuntu 18.04.5 LTS in Section VI-B represents the cloud, the average training time for one message per round is 1.575 ms, resulting in the message throughput rate of approximately 5.67 million messages per hour for the training model. Thus, although there may be some delays in training on the cloud, the average training time for one message per round is only 1.075 ms longer than the average generating time for one message in IVN, which is still at the millisecond level and within an acceptable range. Moreover, the frequency of updating MPMHit is related to the information entropy of the distribution of normal messages in message sequences of the actual IVN. We will study the practical application of the model updating mechanism in the future.

## VII. Conclusion and Future Work

The increase in the attacked surface of ICVs and the lack of security mechanisms in the CAN protocol make IVNs vulnerable to security attacks. This work provides a robust solution to this problem. We propose a new IVN anomaly detection method, IVNSL. A significant advantage of our work over others is that the proposed method can accurately detect message-level anomalies without labels. IVNSL makes the message prediction model learn the distribution of normal messages in sequences in a self-supervised manner. Our proposed message prediction model, MPMHit, can capture both the spatial features of the message and the dependencies between messages. Meanwhile, we propose an online update mechanism for MPMHit based on vehicle-cloud collaboration to solve the concept drift over time. Experimental results show that IVNSL has high Precision, high Recall, high F1-score, and low False Negative Rate in DoS attacks, fuzzy attacks, and spoof attacks. In future work, we will consider compressing the message prediction model to reduce system resource consumption for lightweight deployment and the practical application of the online model update mechanism based on vehicle-cloud collaboration.

## References


[1] K. Agrawal, T. Alladi, A. Agrawal, V. Chamola, and A. Benslimane, "NovelADS: A novel anomaly detection system for intra-vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 22596–22606, Nov. 2022.

[2] M. L. Han, B. I. Kwak, and H. K. Kim, "Event-triggered interval-based anomaly detection and attack identification methods for an in-vehicle network," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2941–2956, 2021.

[3] U. E. Larson, D. K. Nilsson, and E. Jonsson, "An approach to specification-based attack detection for in-vehicle networks," in *Proc. IEEE Intell. Vehicles Symp.*, Jun. 2008, pp. 220–225.

[4] J. Ashraf, A. D. Bakhshi, N. Moustafa, H. Khurshid, A. Javed, and A. Beheshti, "Novel deep learning-enabled LSTM autoencoder architecture for discovering anomalous events from intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4507–4518, Jul. 2021.

[5] X. Duan, H. Yan, D. Tian, J. Zhou, J. Su, and W. Hao, "In-vehicle CAN bus tampering attacks detection for connected and autonomous vehicles using an improved isolation forest method," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2122–2134, Feb. 2023.

[6] H. Qin, M. Yan, and H. Ji, "Application of controller area network (CAN) bus anomaly detection based on time series prediction," *Veh. Commun.*, vol. 27, Jan. 2021, Art. no. 100291. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214209620300620

[7] H. M. Song, H. R. Kim, and H. K. Kim, "Intrusion detection system based on the analysis of time intervals of CAN messages for in-vehicle network," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2016, pp. 63–68.

[8] J. Ning, J. Wang, J. Liu, and N. Kato, "Attacker identification and intrusion detection for in-vehicle networks," *IEEE Commun. Lett.*, vol. 23, no. 11, pp. 1927–1930, Nov. 2019.

[9] B. Groza and P.-S. Murvay, "Efficient intrusion detection with Bloom filtering in controller area networks," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 1037–1051, Apr. 2019.

[10] L. B. Othmane, L. Dhulipala, M. Abdelkhalek, N. Multari, and M. Govindarasu, "On the performance of detecting injection of fabricated messages into the CAN bus," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 468–481, Jan. 2022.

[11] C. V. C. Mille. *Hackers Remotely Kill a Jeep on the Highway—With Me in it*. Accessed: Jul. 21, 2015. [Online]. Available: https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/

[12] W. Wu et al., "A survey of intrusion detection for in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 3, pp. 919–933, Mar. 2020.

[13] A. Zhou, Z. Li, and Y. Shen, "Anomaly detection of CAN bus messages using a deep neural network for autonomous vehicles," *Appl. Sci.*, vol. 9, no. 15, p. 3174, Aug. 2019. [Online]. Available: https://www.mdpi.com/2076-3417/9/15/3174

[14] M. Müter, A. Groll, and F. C. Freiling, "A structured approach to anomaly detection for in-vehicle networks," in *Proc. 6th Int. Conf. Inf. Assurance Secur.*, Aug. 2010, pp. 92–98.

[15] H. Olufowobi, C. Young, J. Zambreno, and G. Bloom, "SAIDuCANT: Specification-based automotive intrusion detection using controller area network (CAN) timing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1484–1494, Feb. 2020.

[16] K.-T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proc. 25th USENIX Conf. Secur. Symp.*, T. Holz and S. Savage, Eds. Austin, TX, USA, Aug. 2016, pp. 911–927. [Online]. Available: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/cho

[17] W. Choi, K. Joo, H. J. Jo, M. C. Park, and D. H. Lee, "VoltageIDS: Low-level communication characteristics for automotive intrusion detection system," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 8, pp. 2114–2129, Aug. 2018.

[18] C. Young, H. Olufowobi, G. Bloom, and J. Zambreno, "Automotive intrusion detection based on constant CAN message frequencies across vehicle driving modes," in *Proc. ACM Workshop Automot. Cybersecurity*. New York, NY, USA: Association for Computing Machinery, Mar. 2019, p. 9, doi: 10.1145/3309171.3309179.

[19] H. Lee, S. H. Jeong, and H. K. Kim, "OTIDS: A novel intrusion detection system for in-vehicle network by using remote frame," in *Proc. 15th Annu. Conf. Privacy, Secur. Trust (PST)*, 2017, pp. 57–5709.

[20] M. Müter and N. Asaj, "Entropy-based anomaly detection for in-vehicle networks," in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2011, pp. 1110–1115.

[21] M. Marchetti, D. Stabili, A. Guido, and M. Colajanni, "Evaluation of anomaly detection for in-vehicle networks through information-theoretic algorithms," in *Proc. IEEE 2nd Int. Forum Res. Technol. Soc. Ind. Leveraging Better Tomorrow (RTSI)*, Sep. 2016, pp. 1–6.

[22] A. R. Javed, S. U. Rehman, M. U. Khan, M. Alazab, and T. Reddy, "CANintelliIDS: Detecting in-vehicle intrusion attacks on a controller area network using CNN and attention-based GRU," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1456–1466, Apr. 2021.

[23] F. Amato, L. Coppolino, F. Mercaldo, F. Moscato, R. Nardone, and A. Santone, "CAN-bus attack detection with deep learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 8, pp. 5081–5090, Aug. 2021.

[24] T.-N. Hoang and D. Kim, "Detecting in-vehicle intrusion via semi-supervised learning-based convolutional adversarial autoencoders," *Veh. Commun.*, vol. 38, Dec. 2022, Art. no. 100520. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214209622000675

[25] S. Longari, D. H. Nova Valcarcel, M. Zago, M. Carminati, and S. Zanero, "CANnolo: An anomaly detection system based on LSTM autoencoders for controller area network," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1913–1924, Jun. 2021.

[26] M. Nam, S. Park, and D. S. Kim, "Intrusion detection method using bi-directional GPT for in-vehicle controller area networks," *IEEE Access*, vol. 9, pp. 124931–124944, 2021.

[27] K. Zhu, Z. Chen, Y. Peng, and L. Zhang, "Mobile edge assisted literal multi-dimensional anomaly detection of in-vehicle network using LSTM," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4275–4284, May 2019.

[28] H. M. Song and H. K. Kim, "Self-supervised anomaly detection for in-vehicle network using noised pseudo normal data," *IEEE Trans. Veh. Technol.*, vol. 70, no. 2, pp. 1098–1108, Feb. 2021.

[29] A. U. Jadhav and N. M. Wagdarikar, "A review: Control area network (CAN) based intelligent vehicle system for driver assistance using advanced RISC machines (ARM)," in *Proc. Int. Conf. Pervasive Comput. (ICPC)*, Jan. 2015, pp. 1–3.

[30] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network," in *Proc. 16th Annu. Conf. Privacy, Secur. Trust (PST)*, Aug. 2018, pp. 1–6.

[31] R. Li, C. Liu, and F. Luo, "A design for automotive CAN bus monitoring system," in *Proc. IEEE Vehicle Power Propuls. Conf.*, Sep. 2008, pp. 1–5.

[32] S. C. Hpl, "Introduction to the controller area network (CAN)," Texas Instrum. Incorporated, Dallas, TX, USA, Tech. Rep., SLOA101, pp. 1–17, 2002.

[33] Bosch GmbH Robert. (1991). *Can Specification 2.0*. [Online]. Available: http://esd.cs.ucr.edu/webres/can20.pdf

[34] P. Cheng, M. Han, and G. Liu, "Des.-IDS: Towards an efficient real-time automotive intrusion detection system based on deep evolving stream clustering," *Future Gener. Comput. Syst.*, vol. 140, pp. 266–281, Mar. 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X22003351

[35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[36] M. Marchetti and D. Stabili, "READ: Reverse engineering of automotive data frames," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 4, pp. 1083–1097, Apr. 2019.

[37] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.* Red Hook, NY, USA: Curran Associates, 2017, pp. 6000–6010.

[38] J. T. Hancock and T. M. Khoshgoftaar, "Survey on categorical data for neural networks," *J. Big Data*, vol. 7, no. 1, pp. 1–41, Dec. 2020, doi: 10.1186/s40537-020-00305-w.

[39] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2016, *arXiv:1606.08415*.

[40] H. M. Song, J. Woo, and H. K. Kim, "In-vehicle network intrusion detection using deep convolutional neural network," *Veh. Commun.*, vol. 21, Jan. 2020, Art. no. 100198. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214209619302451

[41] T. Limbasiya, K. Z. Teng, S. Chattopadhyay, and J. Zhou, "A systematic survey of attack detection and prevention in connected and autonomous vehicles," *Veh. Commun.*, vol. 37, Oct. 2022, Art. no. 100515. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214209622000626

[42] P. Cheng, M. Han, A. Li, and F. Zhang, "STC-IDS: Spatial–temporal correlation feature analyzing based intrusion detection system for intelligent connected vehicles," *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 9532–9561, Nov. 2022. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/int.23012

[43] Y. Zikria, S. Kim, M. Afzal, H. Wang, and M. Rehmani, "5G mobile services and scenarios: Challenges and solutions," *Sustainability*, vol. 10, no. 10, p. 3626, Oct. 2018. [Online]. Available: https://www.mdpi.com/2071-1050/10/10/3626

**Jinhui Cao** (Student Member, IEEE) received the B.S. degree in network engineering from the Changchun University of Science and Technology, China, in 2019, where he is currently pursuing the Ph.D. degree in computer science and technology. His research interests include log mining, network security, anomaly detection, intrusion detection, and artificial intelligence.

**Xiaoqiang Di** (Member, IEEE) received the B.S. degree in computer science and technology and the M.S. and Ph.D. degrees in communication and information systems from the Changchun University of Science and Technology, Changchun, China, in 2002, 2007, and 2014, respectively. From August 2012 to August 2013, he was a Visiting Scholar with the Norwegian University of Science and Technology, Norway. He is currently a Professor and a Ph.D. Supervisor with the Changchun University of Science and Technology. His major research interests include network information security and integrated networks.

**Xu Liu** received the B.E. degree in network engineering from Qingdao University, China, in 2013, and the M.E. degree in computer software theory from the Changchun University of Science and Technology, China, where she is currently pursuing the Ph.D. degree in computer science and technology. Her research interests include network security, information security, anomaly detection, artificial intelligence, big data, game theory, and cloud computing.

**Jinqing Li** received the B.S. degree from the Changchun University of Technology, Changchun, China, in 2002, and the M.S. and Ph.D. degrees from the Changchun University of Science and Technology, Changchun, in 2007 and 2014, respectively. She is currently an Associate Professor and a Master's Student Supervisor with the Changchun University of Science and Technology. Her research interests include network security, information security, and image encryption.

**Zhi Li** received the B.S. degree in computer science and technology from the Shandong University of Finance and Economics in 2021. He is currently pursuing the master's degree in electronic information with the Changchun University of Science and Technology. His research interests include intelligent connected vehicle security, anomaly detection, intrusion detection, and artificial intelligence.

**Liang Zhao** (Member, IEEE) received the Ph.D. degree from the School of Computing, Edinburgh Napier University, in 2011. He is currently a Professor with Shenyang Aerospace University, China. Before joining Shenyang Aerospace University, he was an Associate Senior Researcher with Hitachi (China) Research and Development Corporation from 2012 to 2014. He was also a JSPS invitational Fellow in 2023. He was listed as the Top 2% of scientists in the world by Stanford University in 2022. His research interests include ITS, VANET, WMN, and SDN. He has published more than 150 articles. He was a recipient of the Best/Outstanding Paper Awards from the 2015 IEEE IUCC, 2020 IEEE ISPA, 2022 IEEE EUC, and 2013 ACM MoMM. He served as the Chair of several international conferences and workshops, including the Steering Co-Chair for the 2022 IEEE BigDataSE, the Program Co-Chair for the 2021 IEEE TrustCom, the Program Co-Chair for the 2019 IEEE IUCC, and the Founder of NGDN Workshop (2018–2022). He is an Associate Editor of *Frontiers in Communications and Networking* and *Journal of Circuits Systems and Computers*. He is/has been a Guest Editor of IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING and *Journal of Computing* (Springer).

**Ammar Hawbani** received the B.S., M.S., and Ph.D. degrees in computer software and theory from the University of Science and Technology of China (USTC), Hefei, China, in 2009, 2012, and 2016, respectively. He is currently an Associate Professor with Shenyang Aerospace University, China. His research interests include WSN and WBAN.

**Mohsen Guizani** (Fellow, IEEE) received the B.S. (Hons.), M.S., and Ph.D. degrees in electrical and computer engineering from Syracuse University, Syracuse, NY, USA, in 1985, 1987, and 1990, respectively. He is currently a Professor in machine learning and an Associate Provost at the Mohamed bin Zayed University of Artificial Intelligence (MBZUAI), Abu Dhabi, United Arab Emirates. Previously, he worked in different institutions in USA. He is the author of ten books and more than 800 publications. His research interests include applied machine learning and artificial intelligence, the Internet of Things (IoT), intelligent autonomous systems, smart cities, and cybersecurity. He was listed as a Clarivate Analytics Highly Cited Researcher in Computer Science in 2019, 2020, and 2021. He has won several research awards, including the "2015 IEEE Communications Society Best Survey Paper Award," the Best ComSoc Journal Paper Award in 2021, and five best paper awards from ICC and GLOBECOM Conferences. He was a recipient of the 2017 IEEE Communications Society Wireless Technical Committee (WTC) Recognition Award, the 2018 AdHoc Technical Committee Recognition Award, and the 2019 IEEE Communications and Information Security Technical Recognition (CISTC) Award. He served as the Editor-in-Chief of *IEEE Network* and is currently serving on the editorial boards of many IEEE TRANSACTIONS and magazines. He was the Chair of the IEEE Communications Society Wireless Technical Committee and the Chair of the TAOS Technical Committee. He served as the IEEE Computer Society Distinguished Speaker and is currently the IEEE ComSoc Distinguished Lecturer.